

Almost every element of a chart is customizable, so you can tell Matplotlib to keep using plain notation if you prefer:

```
--snip--
# Set the range for each axis.
ax.axis([0, 1100, 0, 1_100_000])
ax.ticklabel_format(style='plain')

plt.show()
```

The `ticklabel_format()` method allows you to override the default tick label style for any plot.

Defining Custom Colors

To change the color of the points, pass the argument `color` to `scatter()` with the name of a color to use in quotation marks, as shown here:

```
ax.scatter(x_values, y_values, color='red', s=10)
```

You can also define custom colors using the RGB color model. To define a color, pass the `color` argument a tuple with three float values (one each for red, green, and blue, in that order), using values between 0 and 1. For example, the following line creates a plot with light-green dots:

```
ax.scatter(x_values, y_values, color=(0, 0.8, 0), s=10)
```

Values closer to 0 produce darker colors, and values closer to 1 produce lighter colors.

Using a Colormap

A *colormap* is a sequence of colors in a gradient that moves from a starting to an ending color. In visualizations, colormaps are used to emphasize patterns in data. For example, you might make low values a light color and high values a darker color. Using a colormap ensures that all points in the visualization vary smoothly and accurately along a well-designed color scale.

The `pyplot` module includes a set of built-in colormaps. To use one of these colormaps, you need to specify how `pyplot` should assign a color to each point in the dataset. Here's how to assign a color to each point, based on its `y`-value:

```
scatter
_squares.py  --snip--
plt.style.use('seaborn')
fig, ax = plt.subplots()
ax.scatter(x_values, y_values, c=y_values, cmap=plt.cm.Blues, s=10)

# Set chart title and label axes.
--snip--
```

The `c` argument is similar to `color`, but it's used to associate a sequence of values with a color mapping. We pass the list of `y`-values to `c`, and then